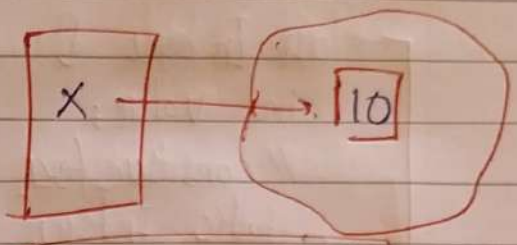
Auto Boxing, Abstract Classes & POJOs

* Autoboxing / unboxing

Data types

primitives	non-primitives
int	Integer
long	Long
short	Short
float	Float
double	Double
char	Character
boolean	Boolean

```
int x = 10;
Integer x = new Integer(10);
```




```
class Integer {
    int value;
    // constructor
    // methods
}
```

Primitives	Wrappers
int	Integer
float	Float
⋮	⋮

① → legacy
② → fast

Autoboxing / Unboxing

int → Integer

```
int x = 10;
Integer y = x; // Auboxing
Integer y = new Integer(x); //old
Integer y = Integer.valueOf(x); //new
```

```
class Integer {
    int value;
    // constructors, Integer
    static valueOf(int x){
    ==
    }
}
```

```
Integer x = 10;
int y = x;          // unboxing
```

```
int y = x.intValue();
```

```
class Integer {
    int value;
    // constructors
    static Integer valueOf(int x) {
        // do something
    }
    int intValue() {
        return value;
    }
}
```

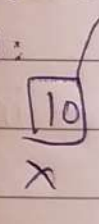
```
int x = 100;
int y = 100;
```
(x == y) // true

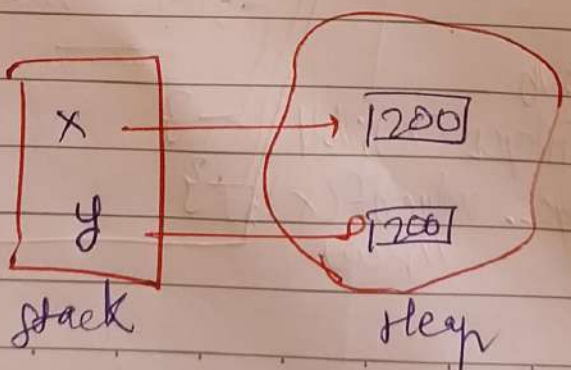
```
Integer x = 200;
Integer y = 200;
```
(x == y) // false

```
Integer x = 200; // Integer y = 200;
```

```
Integer x = Integer.valueOf(200);
x = new Integer(200);
```

```
int x = 10;
int y = 10;
```
(x == y)





(==) → it compares references

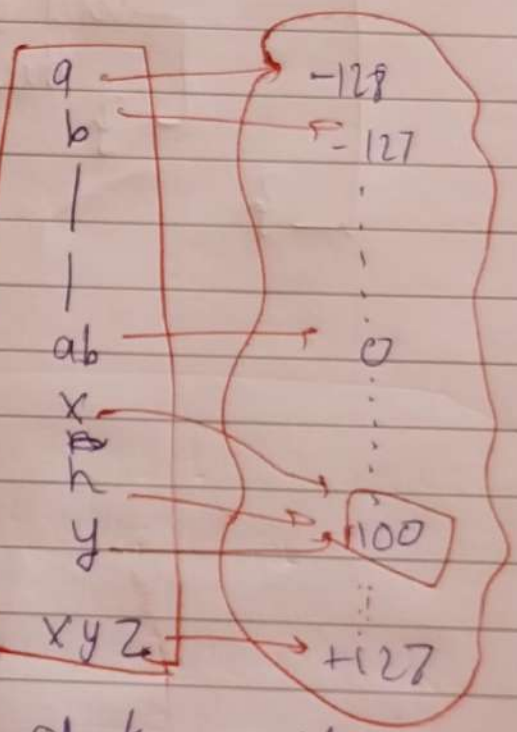
x.equals(y) → true

```
public final class Integer {
    private int value;
    public Integer(int value) {          // Conceptual
        this.value = value;
    }
    public int intValue() {
        return value;
    }
    public static Integer valueOf(int v) {
        // give Object Integer of X
    }
    public boolean equals(Integer y) {
        return (this.value == y.value);
    }
}
```

```
Integer X = 200;
Integer y = 200;
(X == y)  // false
X = 100;
y = 100
(X == y)  // true
```

Integer value of (-128 to +127)

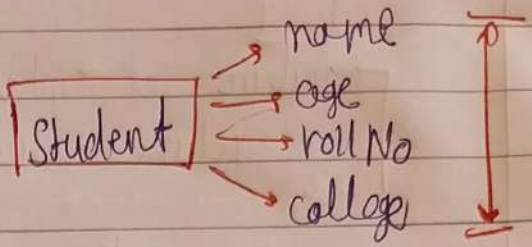
```
Integer X = 100;
X = Integer.valueOf(100);
Integer y = 100;
y = Integer.valueOf(100);
(X == y)
```



-128 — +127

* Abstract Classes
→ Any class which can't be instantiated

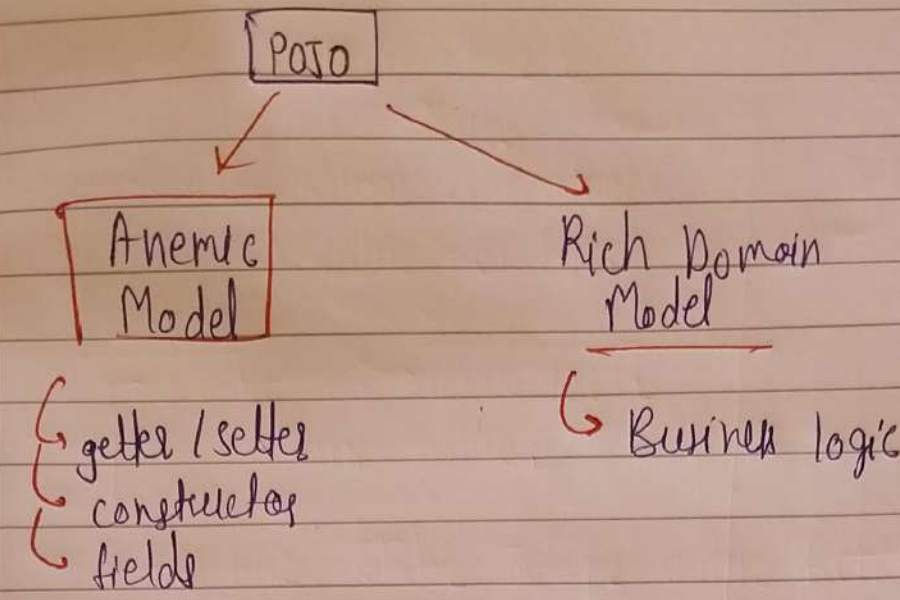
* POJO Classes
→ Plain old Java Object




```
class Sample {
    int x;
    String y;
    Sample (int x, String y) {
    }
    int getX() {
        return x;
    }
    void setX (int x) {
        this.x = x;
    }
}
```

```
class Student {
}
```

POJO
↳ Hard Core Business logic
→ getter / setter
→ Constructor
→ variable (Fields)
→ Builder
→ Business logic




```
class Student {
    String name;
    Student (name){
    }
    // getters
    // setters
}
```

```
markAttendance(){
}
```